AprilTag ID Sorting

AprilTag ID Sorting
1. Course Content
2. Preparation
3. Case Demonstration
4. Core Source Code Analysis

1. Course Content

This feature implements the program acquiring images through the camera and recognizing the AprilTags in the images; the robotic arm lowers its gripper to grasp the AprilTag and, according to the AprilTag's ID, places it in different positions.

The wooden block used in this section: **30x30x30mm Aruco marker wooden block**.

Aruco marker:

- **Border**: the outer black frame, which helps the algorithm quickly locate the marker.
- **Payload (encoding region)**: the inner black and white squares, storing the unique ID (AprilTag).
- **Error Correction**: some Aruco families support error correction, improving the ability to resist occlusion.

Aruco has a variety of predefined "families", and different families have different encoding rules and capacities:

- tag16h5: 4×4 matrix, small, low redundancy
- tag25h9: 5×5 matrix, balancing size and reliability
- tag36h11: 6×6 matrix, high capacity, strong anti-occlusion ability
- tagCircle21h7: circular design, suitable for rotation scenarios

Suffixes such as `h5` and `h9` indicate the Hamming distance (error correction capability); the larger the number, the stronger the fault tolerance.

2. Preparation

```
ros2 launch yahboomcar_bringup arm_demo.launch.py demo:=apriltag_sort
```

3. Case Demonstration

- When an AprilTag appears in the image, the program will recognize it, as shown in the figure below

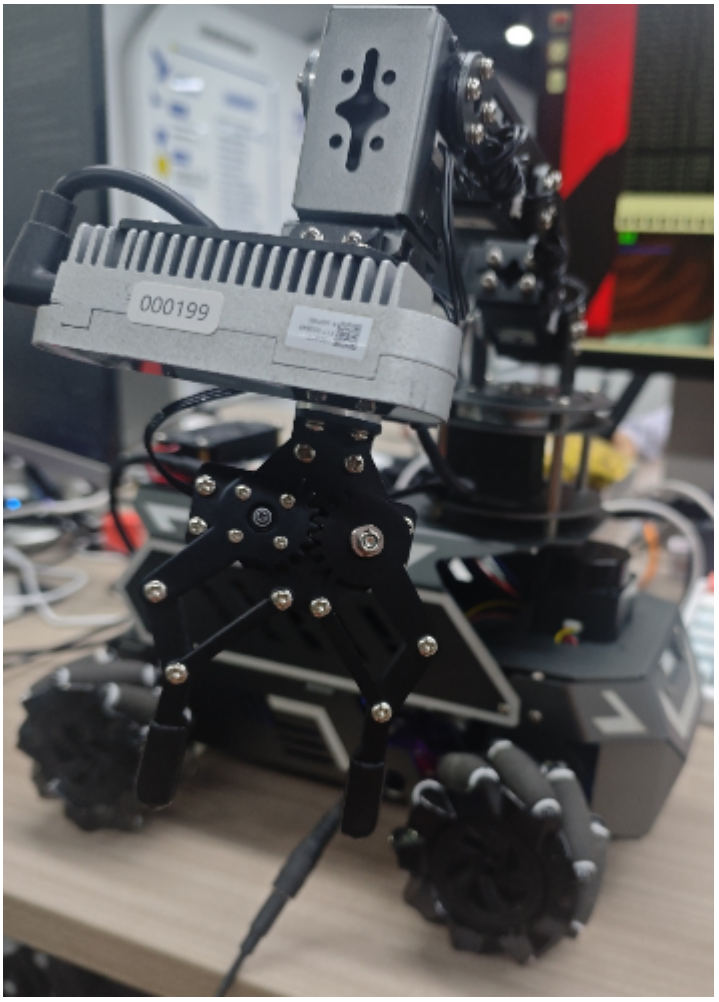
The detection image consists of the following elements:

- **AprilTag marker border**: a green quadrilateral frames the detected AprilTag region, indicating the AprilTag's position in the image

- **ID number**: the integer ID displayed above or inside the AprilTag (such as 1, 2, 3, 4), used to distinguish different targets and determine the placement position
- **Coordinate information**: displays the 3D position of the AprilTag in the camera coordinate frame or the measure coordinate frame (x/y/z, unit: meters)
- **Frame rate (FPS)**: displays the current video frame rate in the upper-left



- Press the spacebar to start the grasping program. The program will choose the nearest block and then grasp it. The chassis will first align with the target. After the chassis alignment with the target is complete, grasping is performed. After grasping is complete, the block is placed in different positions according to the AprilTag number. The robotic arm then resets, and pressing the spacebar again allows the next sorting round.



4. Core Source Code Analysis

- Program path:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/common_demo/common_demo/arm_demo/apriltag_sort.py
```

Here the flow of aligning with the grasping target is explained:

_iterative_align function (iterative alignment):

This function adjusts the chassis position through multiple iterations to make the robot aim at the target AprilTag, ensuring that the relative position between the robotic arm end-effector and the target meets the preset conditions during grasping.

- `tolerance = 0.01`: alignment tolerance (meters); when the position error is less than this value, alignment is considered complete
- `required_stable = 2`: alignment must be detected as stable for 2 consecutive times before returning, preventing a single accidental satisfaction of the condition
- `max_iterations = 12`: maximum number of iterations; after exceeding this, alignment is abandoned and the current pose is returned
- `max_step = 0.04`: maximum distance of a single move (meters), limiting the adjustment amplitude each time to avoid overshoot

- `_get_fresh_tag_data(tag_id)`: obtains the latest AprilTag detection result and pose data
- `delta_x = pose[0] - grasp_distance_x`: calculates the deviation between the current x direction and the expected grasping distance
- `delta_y = pose[1] - grasp_distance_y`: calculates the deviation between the current y direction and the expected grasping distance
- `x_needs` / `y_needs`: determines whether further adjustment in the x/y direction is needed
- `stable_count`: stability counter; increments when the alignment condition is satisfied consecutively, otherwise resets
- `move_by_odom(distance=..., direction=...)`: calls the chassis odometry Action to move the specified distance; the direction is determined by the sign of the deviation (forward/left/right)
- After each move, wait 2 seconds for the chassis to stabilize, then re-detect the pose

_execute_grasp function (execute grasping):

This function is the main control function of the grasping flow, sequentially completing alignment, angle calculation, inverse kinematics solution, robotic arm motion, gripper closing, placement, and other steps.

- `_iterative_align(tag_id)`: calls the iterative alignment function to obtain the aligned AprilTag pose
- `_get_fresh_tag_data(tag_id)`: re-obtains the latest detection result after alignment is complete, and extracts the AprilTag's yaw angle (yaw)
- `normalize_angle_to_0_45(rvec_yaw_deg)`: normalizes the yaw angle to the range 0°~45°, used to determine the gripper rotation direction
- `grasp_offset`: grasping offset `[gx, gy, gz, pitch]`, compensating for the difference between the camera-detected position and the actual grasping position
- `arm_kinematics.inverse_kinematics(...)`: calls the inverse kinematics solver, taking the target position (camera pose + grasping offset) and pitch angle as input, and outputting 5 joint angles
- `result.approximate`: marks whether the solution is approximate; if so, outputs a position error warning
- `result.adjusted_pitch`: if the solver automatically adjusted the pitch angle, outputs the adjusted value
- `joints[0:3] = round(ik_joints[0:3])`: rounds the J1~J4 joint angles obtained from the inverse solution to integers (the low-level firmware only accepts integers)
- `joints[4] = int(90 + angle_normalized)`: the J5 joint angle is calculated from the normalized yaw angle, so that the gripper rotation aligns with the AprilTag direction
- `joints[5] = 30`: the J6 gripper angle is set to 30° (open state)
- `pubSix_Arm(joints)`: publishes the 6-joint control command, and the robotic arm moves to the grasping pose
- `SingleJoint_pub.publish(ArmJoint(id=6, joint=gripping_angle, time=1500))`: controls the gripper to close individually, where `gripping_angle` is the grasping angle
- According to `tag_id` (1~4), selects a different placement joint angle `down_joint`, and places the AprilTag at the corresponding position
- `SingleJoint_pub.publish(ArmJoint(id=6, joint=30, time=1500))`: opens the gripper to release the

object

- `pubSix_Arm(observation_pose)`: returns the robotic arm to the observation pose to prepare for the next sorting

```
    def _iterative_align(self, tag_id):
        tolerance = 0.01
        required_stable = 2
        max_iterations = 12
        max_step = 0.04
        stable_count = 0
        for iteration in range(max_iterations):
            if not rclpy.ok():
                return None

            time.sleep(0.5)
            tag, pose, _ = self._get_fresh_tag_data(tag_id)
            if tag is None or pose is None:
                self.get_logger().warn(f"[Iter {iteration+1}] tag {tag_id} not visible")
                stable_count = 0
                continue

            delta_x = pose[0] - self.grasp_distance_x
            delta_y = pose[1] - self.grasp_distance_y
            x_needs = delta_x > tolerance
            y_needs = abs(pose[1]) > self.grasp_distance_y + tolerance

            if not x_needs and not y_needs:
                stable_count += 1
                if stable_count >= required_stable:
                    self.get_logger().info("Chassis alignment stable, returning latest pose
for IK")
                    return pose
                time.sleep(0.3)
                continue
            stable_count = 0

            if x_needs:
                move_x = min(delta_x, max_step)
                self.move_by_odom(distance=move_x, direction='forward')

            if y_needs:
                move_y = min(abs(pose[1]) - self.grasp_distance_y, max_step)
                direction = 'left' if pose[1] > 0 else 'right'
                self.move_by_odom(distance=move_y, direction=direction)
            time.sleep(2.0)

        self.get_logger().warn(f"Alignment did not converge after {max_iterations}
iterations")
        _, pose, _ = self._get_fresh_tag_data(tag_id)
        return pose

    def _execute_grasp(self, tag_id):
```

```
        self.get_logger().info(f"Grasp thread started for tag ID {tag_id}")
        final_pose = self._iterative_align(tag_id)
        if final_pose is None:
            self.get_logger().error("Iterative alignment failed")
            self._reset_grasp_state()
            return

        time.sleep(0.5)
        latest_tag, _, rvec_yaw_deg = self._get_fresh_tag_data(tag_id)
        if latest_tag is None:
            self.get_logger().error("Tag lost after alignment")
            self._reset_grasp_state()
            return

        angle_normalized = normalize_angle_to_0_45(rvec_yaw_deg)
        self.get_logger().info(f"Tag rvec yaw: {rvec_yaw_deg:.1f}° -> normalized:
{angle_normalized:.1f}°")
        mx, my, mz = final_pose
        gx, gy, gz = self.grasp_offset[0], self.grasp_offset[1], self.grasp_offset[2]
        self.get_logger().info(f"camera measure pose : x={mx:.3f}, y={my:.3f}, z={mz:.3f};IK
target with grasp_offset: x={mx+gx:.3f}, y={my+gy:.3f}, z={mz+gz:.3f}, pitch=
{self.grasp_offset[3]:.1f}°")

        result: KinResult = self.arm_kinematics.inverse_kinematics(
            target_x=mx + gx,
            target_y=my + gy,
            target_z=mz + gz,
            target_pitch=self.grasp_offset[3],
            seed_angles_deg=None,
        )

        if not result.success:
            self.get_logger().error(f"Grasp IK failed: {result.message}")
            self._reset_grasp_state()
            return

        if result.approximate:
            self.get_logger().warn(Fore.YELLOW+f"IK using approximate solution, position
error = {result.position_error:.4f} m"+Fore.RESET)
        if result.adjusted_pitch is not None:
            self.get_logger().warn(Fore.YELLOW+f"IK using pitch sweep, adjusted pitch =
{result.adjusted_pitch:.1f}°"+Fore.RESET)

        ik_joints = result.data
        joints = [0, 0, 0, 0, 0, 0]
        joints[0] = round(ik_joints[0])
        joints[1] = round(ik_joints[1])
        joints[2] = round(ik_joints[2])
        joints[3] = round(ik_joints[3])
        joints[4] = int(90+angle_normalized)
        joints[5] = 30

        self.get_logger().info(f"Grasp joints: {joints}")
```

```
        self.pubSix_Arm(joints)
        time.sleep(3.5)
        self.SingleJoint_pub.publish(ArmJoint(id=6, joint=self.gripping_angle, time=1500))
        time.sleep(2.0)

        if latest_tag.tag_id == 1:
            down_joint = [180, 35, 70, 5, 60,self.gripping_angle]
        elif latest_tag.tag_id == 2:
            down_joint = [180, 35, 60, 0, 90,self.gripping_angle]
        elif latest_tag.tag_id == 3:
            down_joint = [0, 30, 70, 2, 90,self.gripping_angle]
        elif latest_tag.tag_id == 4:
            down_joint = [0, 43, 48, 6, 90,self.gripping_angle]
        self.pubSix_Arm(down_joint)
        time.sleep(2.5)
        self.SingleJoint_pub.publish(ArmJoint(id=6, joint=30, time=1500))
        time.sleep(2.0)
        self.pubSix_Arm(self.observation_pose)
        time.sleep(2.5)
        self._reset_grasp_state()
```

- Parameter file

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_bringup/param/common.yaml
```

- When there is a grasping deviation, the grasp_offset can be adjusted

```
apriltag_sort:
  ros__parameters:
    apriltag_grasp_distance: [0.1, 0.0]     # x,y axis alignment distance (meters)
    tag_size: 0.02                          # AprilTag side length (meters)
    grasp_offset: [0.00,0.0,0.015,85.0]     # x,y,z axis offset distance and included angle,
compensating for the error reaching the target point caused by servo backlash and link
rigidity degradation
    gripping_angle: 140                     # gripper grasping angle, in degrees, integer
    observation_pose: [90,107,-1,0,90,30]
```